

Project Proposal: AutoDoc AI

1. Introduction

Background

In most software projects, documentation is often an afterthought. Developers are usually preoccupied with writing code, fixing issues, and deploying features—leaving little room for properly documenting the system. Over time, this leads to incomplete or outdated documentation, which makes onboarding, collaboration, and maintenance more difficult.

Objective

This project proposes an AI-driven solution that automates the generation of core documentation artifacts for GitHub repositories. The goal is to minimize manual effort while improving the quality and consistency of documentation. The tool will generate structured README files, interactive tutorials, and architecture diagrams based on the codebase of the repository.

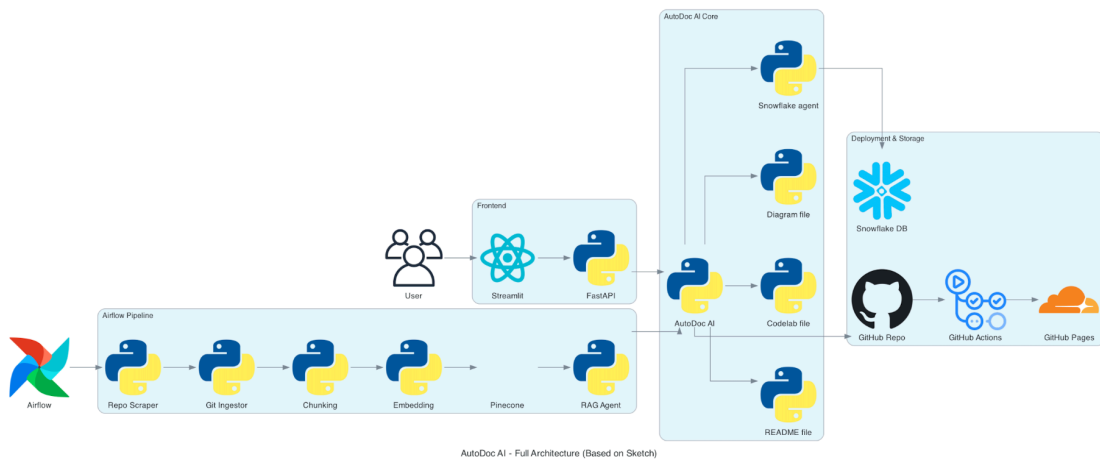
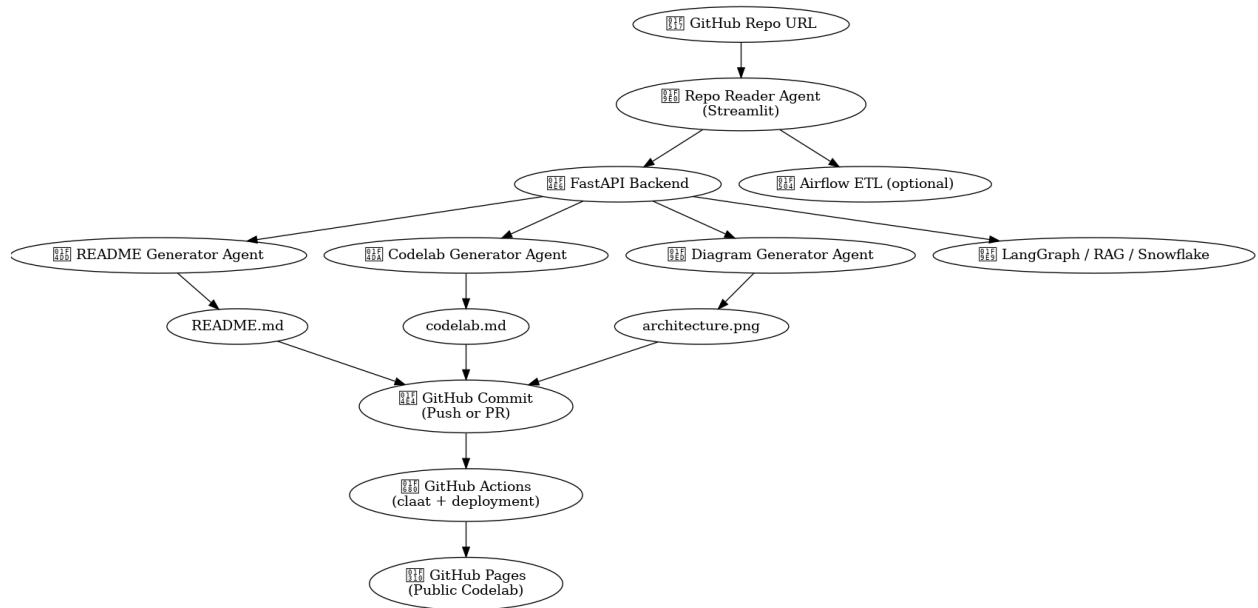
Solution Summary

AutoDoc AI is a web-based application built with Streamlit that allows users to input a GitHub repository URL and receive an automatically generated:

- `README.md` file summarizing project purpose, usage, and structure
- Interactive Codelab tutorial created using the Claat framework
- Architecture diagram based on code structure and dependencies
- GitHub integration to commit or push documentation files directly
- Publicly shareable Codelab tutorial link

All content is generated using Large Language Models (LLMs) and autonomous agents, offering a complete documentation solution with minimal developer input.

2. Project Scope and Architecture



Data Sources

- **Unstructured:** GitHub repositories containing code and minimal metadata

- **Structured:** Project-specific metadata stored in databases (PineconeDB, Snowflake)

Technologies Used

- **Frontend:** Streamlit
- **Backend:** FastAPI
- **Agent Framework:** LangGraph
- **Language Model:** GPT-4o mini
- **Data Orchestration:** Apache Airflow
- **Storage:** AWS S3
- **CI/CD:** GitHub Actions
- **Diagram Generation:** Mermaid, Mingrammer (Diagrams-as-Code tools)
- **Tutorial Generation:** Google Claat
- **Database/Vector Store:** PineconeDB, Snowflake

Expected Outputs

- Auto-generated **README .md** files
 - Interactive tutorial links based on Codelabs
 - Visual architecture diagrams
 - Cloud-hosted deployment of the application
 - Testing strategies (unit and integration)
 - Video walkthrough of the project
 - CI/CD-enabled GitHub repo
-

3. Problem Statement

Key Challenges

- **Time-Intensive Documentation:** Developers find it difficult to allocate time to write detailed documentation for each stage of the development lifecycle.
 - **Limited Codebase Understanding:** Contributors often face delays due to lack of context or architectural clarity when exploring unfamiliar repositories.
 - **Out-of-Sync Documentation:** Documentation quickly becomes outdated when not actively maintained in parallel with the code.
-

4. Project Advantages

Efficiency and Time-Saving

The tool significantly reduces the time developers spend on documentation by automating most of the process.

Version-Safe Integration

Automatic push or pull request generation ensures that documentation is always synchronized with the repository.

Improved Project Adoption

Projects with clear and professional documentation are easier to understand and adopt, encouraging external contributions and use.

Reusable and Scalable

AutoDoc AI can be deployed across multiple repositories, making it valuable for organizations looking to enforce standard documentation practices.

5. Implementation Methodology

Workflow Pipeline

1. **Data Extraction:** Use BeautifulSoup to gather repository data from GitHub.
2. **Code Conversion:** Utilize Gitingest to transform code files into structured text.
3. **Chunking:** Apply a Recursive Token Chunker to prepare data for embeddings.
4. **Embedding:** Generate vector embeddings using `all-MiniLM-L6-v2`.
5. **Retrieval-Augmented Generation (RAG):** Feed embedded content to GPT-4o mini using LangGraph agents.
6. **Visualization:** Use Mermaid or Mingrammer to generate diagrams from parsed structure.
7. **Deployment:** Automate using GitHub Actions and store processed files in AWS S3.
8. **Tutorial Hosting:** Generate tutorials with Claat and host them using GitHub Pages.

6. Timeline and Milestones

Phase	Task	Deliverables	Timeline
Phase 1	Initial extraction and embedding pipeline	Airflow DAG and PineconeDB population	April 5–8
Phase 2	Agent implementation and documentation generation	README, diagrams, Codelabs	April 9–12
Phase 3	GitHub integration and cloud deployment	Fully documented repo with live Codelab	April 13–18

7. Roles and Responsibilities

- **Asavari:** MCP Server integration, backend (FastAPI), and frontend (Streamlit)
- **Hishita:** GitHub automation for commits and pull requests

- **Pratik:** Airflow pipeline development and agentic AI workflow integration
-

8. Risks and Mitigation Strategies

Risk	Issue	Mitigation
Incorrect AI Output	LLMs may misunderstand complex logic	Incorporate validation layers and human review checkpoints
LLM Constraints	Some codebases may be too large or ambiguous	Use modular prompts, file-wise summarization, and chunking
Privacy Concerns	Handling of private repos requires security measures	Use encrypted access tokens and secure storage protocols

9. Expected Outcomes

- A GitHub repository containing detailed, AI-generated documentation
 - A shareable, interactive Codelab tutorial
 - An architecture diagram matching the project's technical structure
 - CI/CD-enabled updates through GitHub Actions
 - A deployed solution that can be reused for other repositories
-

10. Conclusion

AutoDoc AI streamlines documentation by applying artificial intelligence to one of the most time-consuming aspects of development. It allows developers to:

- Automatically produce consistent and informative README files
- Generate interactive tutorials without manual effort

- Visualize code architecture from source structure
- Commit documentation directly to GitHub with minimal setup